

# Python ile Nesne Yönelimli Programlama (Object oriented programming with Python)

Mehmet Hakan Satman (PhD.)

April 15, 2022

# İçindekiler

|     |                                           |    |
|-----|-------------------------------------------|----|
| 1.1 | Class (Sınıf) . . . . .                   | 2  |
| 1.2 | Parabol için bir sınıf denemesi . . . . . | 4  |
| 1.3 | Vector2D Sınıfı . . . . .                 | 7  |
| 1.4 | Shape sınıfı . . . . .                    | 9  |
| 1.5 | Zincirleme Miras . . . . .                | 11 |

## 1.1 Class (Sınıf)

`list` veri yapısının bir sınıf, herhangi bir listenin de bu sınıfın bir örneği (object) olduğu söylenebilir.

```
a = [1, 2, 3, 4, 10, 90, 100]
print(type(a))
```

Programı çalıştırıldığında:

```
<type 'list'>
```

`a` değişkeninin `list` tipinde olduğu görülebiliyordu. Burada `a` bir `list` olmanın yanında aynı zamanda `list` sınıfından (class) örneklenirilmiş bir nesnedir (object).

Python'da `dir` fonksiyonu ile bu nesnenin sahip olduğu değişken ve fonksiyon isimlerine erişilebilir:

```
>>> dir(a)
['__add__', '__class__', '__contains__', '__delattr__', '__delitem__',
 '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__',
 '__getitem__', '__gt__', '__hash__', '__iadd__', '__imul__', '__init__',
 '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__',
 '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',
 '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__',
 '__str__', '__subclasshook__', 'append', 'clear', 'copy', 'count',
 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
```

Çıktıdan görüldüğü üzere `a`'nın `insert`, `pop`, `remove` ve `sort` gibi üyeleri vardır. Bu üyeler bir değişken olabileceği gibi bir fonksiyon da olabilir. Burada sayılan fonksiyonlar, `a` nesnesi üzerinden nokta (dot) operatörü ile çağırılabilirler:

```
>>> a
[1, 2, 3, 4, 10, 90, 100]
>>> a.pop()
100
>>> a
[1, 2, 3, 4, 10, 90]
>>> a.pop()
90
>>> a
[1, 2, 3, 4, 10]
>>> a.pop()
10
>>> a
[1, 2, 3, 4]
```

```
[1, 2, 3, 4]
>>> a.insert(1, -1000)
>>> a
[1, -1000, 2, 3, 4]
```

Görüldüğü gibi `pop` fonksiyonu ile listenin sonundaki elemana ulaşılmış, aynı zamanda bu eleman listeden çıkarılmıştır. Benzer şekilde `insert` fonksiyonu ile `a`'nın birinci elemanı olarak `-1000` değeri yerleştirilmiştir. Python'da indisler 0'dan başladığı için ilk eleman listenin sıfırıncı elemanıdır. Bu bağlamda listenin 1 indisli elemanı aslında listenin ikinci elemanına karşılık gelir.

Burada dikkat edilmesi gereken nokta listeye ait bir işlevin liste üzerinden çağırılıyor olduğudur. Bunun tersi de mümkün olabilirdi. Örneğin:

```
pop(a)
pop(a)
pop(a)
insert(a, 1, -1000)
```

şeklinde çalışan ve aynı etkiyi veren fonksiyonlar yazılabilirdi. Burada dikkat edilmesi gereken, `pop` ve `insert` fonksiyonlarının listeye ait olmadığı ve hepsinin argüman olarak bir liste aldığıdır. Oysa ki nesne yönelimli yaklaşımla bu kodun

```
nesne.fonksion(arguman)
```

şeklinde yazılabilmesi sağlanmıştır. Tüm bunların sonucunda söyleyebiliriz ki Python sınıfları örneklerine ait değişken ve fonksiyonları **sarmalar** (Encapsulation). Bu terim hem fonksiyon ve değişkenlerin ilgili sınıf tarafından içerildiği anlamına gelir, hem de, bu fonksiyon ve değişkenlere yalnızca ilgili sınıftan türetilen nesneler üzerinden erişilebileceği anlamına gelir.

## 1.2 Parabol için bir sınıf denemesi

Her parabolün (ikinci dereceden polinom fonksiyon) belirli bir düzen izlediğini söyleyebiliriz. Örneğin  $x$  bağımsız değişkeni için bir parabolün

$$f(x) = ax^2 + bx + c$$

şeklinde tanımlandığını söyleyebiliriz.  $a \neq 0$  için farklı  $a$ ,  $b$  ve  $c$  değerleriyle oluşturulmuş ve bu kalıba uyan tüm fonksiyonların bir parabol olduğunu söyleyebiliriz. Nesne yönelimli programlama terimleriyle konuşursak, her parabol, parabol sınıfından örneklenmiş bir nesnedir. Tabi ki bu bir programlama paradigmasıdır ve bu tanım yalnızca ilgili paradigmayı bağlar. Başka bir paradigma, örneğin fonksiyonel programlama, terimleriyle konuşulduğunda bu tanımlar değişebilir.

Şimdi Parabol sınıfı için Python kodunu oluşturalım:

```
class Parabol:

    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c
```

Görüldüğü gibi tüm sınıflar `class` anahtar kelimesi ile tanımlanır. Sınıfların içerdiği elemanlar, `class` direktifinden sonra girintiyle yazılır. Bu girinti sayesinde yazılacak olan tanımların ilgili sınıfa ait olduğu belirtilmiş olur.

Bir sınıftan bir nesne örneklenir örneklenmez özel bir fonksiyonun Python tarafından tetiklenmesi sağlanır. Bu fonksiyon `__init__` adını alır. Nesne oluşur oluşmaz çağırıldığı ve bazı başlangıç işlerinin yürütüldüğü fonksiyon olması bakımından `__init__`'e **constructor** (yapıcı) adı verilir.

Bu fonksiyonun ilk argümanı `self` ile adlandırılmıştır. `self`, bu sınıftan örneklenilecek olan her nesnenin kendisine işaret eder. Sonraki argümanlar `a`, `b` ve `c` ile isimlendirilmiştir ve bir parabolü tanımlamak için gerekli ve yeterli bilgiyi içerirler.

Yapıcı fonksiyon içinde `self.a = a` gibi bir atama, dışarıdan fonksiyona argüman olarak gönderilen `a` değerinin oluşan nesnede saklanacağı anlamına gelir. Şimdi bu parabol sınıfından bir nesne örnekleyelim:

```
p1 = Parabol(-2, 5, 10)
print(p1)
```

Bu kod çalıştırıldığında aşağıdakine benzer bir çıktı elde edilir:

```
<__main__.Parabol object at 0x107763950>
```

Çıktıdan Parabol'un bir `object` olduğu görülür. `0x1077..` ile verilen bu nesnenin bellekte kapladığı alanın başlangıç adresidir. Şimdilik bununla ilgilenmiyoruz.

Bir parabol  $a$ ,  $b$  ve  $c$  sabitleriyle tanımlanmıştır. Öte yandan parabol ile yapılabilecek bir takım işlemler de vardır. Örneğin bu parabolün varsa köklerini aramak isteyebilirdik. Ya da tepe veya dip noktası araştırılabilirdi. Bu işlevleri Parabol sınıfından örneklendirilmiş nesneler üzerinde çalışacak şekilde tanımlayabiliriz.

Aşağıdaki kodu inceleyelim:

```
class Parabol:

    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def roots(self):
        delta = self.b * self.b - 4 * self.a * self.c
        x1 = (-self.b + (delta ** 0.5)) / (2 * self.a)
        x2 = (-self.b - (delta ** 0.5)) / (2 * self.a)
        return (x1, x2)
```

`roots` fonksiyonu sahip olduğu girintiyle `Parabol` sınıfında tanımlanmıştır. Ayrıca `self` argümanını alarak bu sınıftan örneklendirilmiş her nesnede bu nesnenin tüm üyelerine ulaşabilecek şekilde tanımlandığı görülür. `delta` değişkeni `roots` fonksiyonunun kapsamı içinde geçerlidir ve

$$\Delta = b^2 - 4ac$$

tanımına uygun şekilde yazılmıştır. Burada `b` yerine `self.b` yazılması, bu nesneye ait `b` değişkeninin kullanılacağı anlamına gelir. `self.b`, nesne oluşurken dış dünyadan aktarılan argüman değerinden kopyalanmıştır. Sonrasında `x1` ve `x2` değişkenleri fonksiyon kapsamında kalmakla birlikte

$$x_1 = \frac{-b + \sqrt{\Delta}}{2a}$$

ve

$$x_2 = \frac{-b - \sqrt{\Delta}}{2a}$$

tanımlarına uygun şekilde yazılmıştır. Fonksiyon ilk elemanın  $x_1$  ve ikinci elemanın  $x_2$  olduğu bir `tuple` nesnesi döndürür. Bu sınıfı aşağıdaki şekilde kullanabilirdik:

```
p1 = Parabol(-2, 5, 10)
print(p1.roots())
```

Program çalıştırıldığında aşağıdaki çıktı elde edilir:

```
(-1.3117376914898995, 3.8117376914898995)
```

Son olarak da, aynı temel kavramları kullanarak, Parabol sınıfını tepe noktasını döndürecek bir fonksiyon ekleyerek şimdilik sonlandıralım:

```
class Parabol:

    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def roots(self):
        delta = self.b * self.b - 4 * self.a * self.c
        x1 = (-self.b + (delta ** 0.5)) / (2 * self.a)
        x2 = (-self.b - (delta ** 0.5)) / (2 * self.a)
        return (x1, x2)

    def maxOrMin(self):
        return -self.b / (2 * self.a)

p1 = Parabol(-2, 5, 10)
print(p1.maxOrMin())
```

Program çalıştırıldığında aşağıdaki gibi bir çıktı elde ederiz:

```
1.25
```

Böylece  $x = 1.25$  noktasında bir maksimum veya minimuma sahip olduğunu söyleyebiliriz. Söz konusu parabolün  $a$  sabiti negatif olduğundan bulduğumuz bir global maksimumdur.

## 1.3 Vector2D Sınıfı

İki boyutlu uzayda yer alan bir vektörleri kapsayacak şekilde bir `Vector2D` sınıfı yazmak istemiş olalım:

```
class Vector2D:

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Vector2D({self.x}, {self.y})"

v1 = Vector2D(0, 5)
print(v1)
```

`Vector2D`, `Parabol` sınıfından farklı olarak `__str__` isimli bir fonksiyon tanımlar. Bu özel fonksiyon `Vector2D` sınıfından üretilmiş bir nesnenin string olarak ele alınması durumunda otomatik olarak çağırılır. Diğer bir deyişle söz konusu fonksiyon, bir `Vector2D`'nin nasıl string veri tipine dönüştürülmesi gerektiğini tanımlar. Sonrasında `Vector2D` sınıfından `v1` adlı bir nesne oluşturulmuştur. `print` fonksiyonu ile bu vektörün bir string gösteriminin yazılması sağlanır. Program çalıştırıldığında aşağıdaki çıktı elde edilir:

```
Vector2D(0, 5)
```

Python, bize ait olan `Vector2D` sınıfının içerdiği ve biz tarafından yazılmamış olan bir takım fonksiyonları otomatik olarak dahil eder. Bunun sebebi, her yazdığımız sınıfın açıkça belirtilse de belirtilmese de `object` isimli bir sınıftan miras alınmasıdır (interitance). `Vector2D` sınıfını gerek olmasa da şöyle de tanımlayabilirdik:

```
class Vector2D(object):

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Vector2D({self.x}, {self.y})"

v1 = Vector2D(0, 5)
print(v1)
```



object sınıfından miras alınarak yazılan Vector2D, başka bir tabirle, object adlı sınıfı genişletir. Böylece object içinde tanımlanmış fonksiyonların aynı zamanda Vector2D’de de tanımlanmış gibi kullanılabilmesine olanak sağlanmış olur. Bunun yanında object sınıfını genişleten Vector2D, kendine ait olan yeni fonksiyonları da tanımlayabilir. init ve str gibi başında çift tire işareti barındıran bu özel fonksiyonların biz yazmış olmasak da aynı zamanda Vector2D’den de erişilebiliyor olmasının sebebi budur.

object sınıfında tanımlanmış olan init ve str gibi fonksiyonları Vector2D yeniden tanımladığı için, bu tanımları bir nevi ezer (override).

Özel \_\_add\_\_ fonksiyonunu yeniden tanımlayarak bir Vector2D ile başka bir Vector2D’nin toplatılmasını ve sonucun yine bir Vector2D olarak döndürülmesini sağlayabiliriz. Aşağıdaki kodu inceleyelim:

```
class Vector2D(object):

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Vector2D({self.x}, {self.y})"

    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)

v1 = Vector2D(0, 5)
v2 = Vector2D(2, 3)
v3 = v1 + v2
print(v3)
```

Program çalıştırıldığında aşağıdaki sonuç elde edilir:

```
Vector2D(2, 8)
```

add fonksiyonu ile başka bir Vector2D nesnesinin argüman olarak alınması ve mevcut Vector2D nesnesiyle toplanması sağlanmıştır. Bir nevi + operatörünün Vector2D için nasıl davranması gerektiğinin tanımlamasını sunan bu işlem *operatör aşırı yükleme* (operator overloading) adını alır.

## 1.4 Shape sınıfı

Geometrik bir şekli tanımlayan Shape sınıfının alan ve çevre bulmak için bir takım fonksiyonlara sahip olduğunu düşünelim:

```
class Shape:

    def çevre(self):
        pass

    def alan(self):
        pass
```

Tanımda yer alan `pass` anahtar kelimesi, fonksiyonların yalnızca tanım bilgisi ile yer alması için kullanılmıştır. Şimdi Shape sınıfını miras alarak Rectangle (Dikdörtgen) sınıfını tanımlayalım:

```
class Rectangle(Shape):

    def __init__(self, a, b):
        self.a = a
        self.b = b

    def çevre(self):
        return 2 * (self.a + self.b)

    def alan(self):
        return self.a * self.b
```

Şimdi de kare için Square adlı sınıfı tanımlayalım. Kare, dikdörtgenin özel hali olduğundan Square sınıfını Shape yerine Rectangle sınıfını miras alarak yazabiliriz:

```
class Square(Rectangle):

    def __init__(self, a):
        Rectangle.__init__(self, a, a)
```

Görüldüğü gibi Square sınıfının yapıcısı (constructor) tek bir *a* argümanı alır. Ancak miras aldığı Rectangle sınıfının *a* ve *b* gibi iki kenara karşılık gelen argümanlar alması gerekir. Square sınıfından bir nesne oluşur oluşmaz, üst sınıfın yapıcısı çağırılır ve Rectangle sınıfının bir örneğinin kısa ve uzun kenarların *a* uzunluğunda olması sağlanır. Aslında Square sınıfının yaptığı pek bir işlem yoktur. Yalnızca özelliklerini miras aldığı bir Rectangle nesnesi gibi davranır. Square için bir çevre ve alan fonksiyonu tanımlanmadığına dikkat edilmelidir. Bunun sebebi, dikdörtgenin *a* ve *b* kenarları için çalışan çevre ve alan fonksiyonlarının kenarları *a* ve *a* olan kare için de çalışması beklenir. Bir kenarı 3 cm olan bir kareyi aşağıdaki şekilde oluşturabiliriz:

```
r = Square(3)
print(r.cevre())
print(r.alan())
```

Program çalıştırıldığında aşağıdaki gibi bir çıktı elde edilir:

```
12
9
```

## 1.5 Zincirleme Miras

Aşağıdaki gibi tanımlanmış A, B ve C sınıflarını ele alalım:

```
class A:
    def func(self):
        pass

class B(A):
    def func(self):
        pass

class C(B):
    def func(self):
        return("func is defined in class C")

myobject = C()
print(myobject.func())
```

Örnekte B, A'yı; C de B'yi miras alarak tanımlanır. `myobject` ise C sınıfından türetilen bir nesnedir. `myobject` üzerinden `func` adlı fonksiyon çağırılırsa ne olur?

Sonuç aşağıdaki gibi olacaktır:

```
func is defined in class C
```

Bunun sebebi C'nin söz konusu fonksiyonu tanımlamış olduğudur. Bu sonuçtan hareketle

```
myobject = B()
print(myobject.func())
```

kodunun `None` yazdıracağını söyleyebiliriz. Çünkü hem B hem de genişlettiği A, bu fonksiyonun gövdesini tanımlamaz.

Şimdi bu durumun benzerini ele alalım:

```
class A:
    def func(self):
        return "func is defined in class A"

class B(A):
    pass

class C(B):
    pass

myobject = C()
print(myobject.func())
```

Örnekte B A'yı, C de B'yi genişletir. Ancak B ve C genişletmekten öte hiçbir ek işlev tanımlamaz. myobject nesnesi ise C'den örneklendirilmiştir. Kendisine ait bir **func** fonksiyonu olmayan C söz konusu çağırıda nasıl davranır?

Sonuç aşağıdaki gibi olacaktır:

```
func is defined in class A
```